

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - AĞ TABANLI PARALEL DAĞITIM
SİSTEMLERİ**

PROJE - 2

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Şevval Ensarioğlu - 22290742

<https://github.com/SevvalEnsarioğlu/blm4522-proje2-db-backup-and-recovery>

İçindekiler

1 Giriş	3
1.1 Projenin Amacı	3
1.2 Kullanılan Araçlar ve Teknolojiler	3
2 Sistem Mimarisi ve Altyapı Tasarımı	4
2.1 Şema ve Tablo Yapısı	4
2.2 Altyapı Kurulumu	4
2.3 backup_record Tablosu	5
3 Yedekleme Stratejileri	6
3.1 Strateji Tanımları	6
4 Yardımcı Fonksiyonlar	7
4.1 get_strategy_id() Fonksiyonu	7
4.2 open_backup_record() Fonksiyonu	7
4.3 take_row_snapshot() Fonksiyonu	8
5 Tam Yedekleme	10
6 Artık ve Fark Yedekleme	12
6.1 Artık Yedekleme	12
6.2 Fark Yedekleme	12
6.3 Artık ve Fark Yedek Karşılaştırması	13
7 Otomasyon — pgAgent Job Tanımlamaları	14
7.1 Job Özet Tablosu	14
7.2 Haftalık Tam Yedek Job	14
7.3 Artık Yedek Job	14
7.4 Fark Yedek Job	15
8 Felaketten Kurtarma Senaryoları	16
8.1 Senaryo A: Kazayla Veri Silme	16
8.2 Senaryo B: DROP TABLE Simülasyonu	16

8.3 Senaryo C: Veri Bozulması	17
9 Point-in-Time Recovery Simülasyonu	18
9.1 PITR Testi	19
10 Yedek Doğrulama Testleri	20
10.1 ROW COUNT Testi	20
10.2 CHECKSUM Testi	20
10.3 RESTORE TEST	20
10.4 Doğrulama Testleri Özeti	21
11 Test ve Demo Sorguları	22
11.1 Yedek Alma Testleri	22
11.2 Felaket Senaryosu Kontrolleri	22
11.3 Doğrulama Sonuçları	23
12 Sonuç	24

1 Giriş

Bu proje kapsamında PostgreSQL üzerinde çalışan Northwind veritabanı için yedekleme ve felaketten kurtarma planı tasarlanmıştır. Çalışmada tam yedek, artık yedek ve fark yedek stratejileri oluşturulmuş; bu yedeklerin zamanlayıcılarla otomatik alınması, felaket senaryolarının simüle edilmesi, point-in-time recovery mantığının uygulanması ve alınan yedeklerin doğrulanması hedeflenmiştir.

Proje, SQL Server tarafındaki Backup, Point-in-time Restore ve Database Mirroring gibi kavramlara karşılık gelecek şekilde PostgreSQL ortamında modellenmiştir. Bu nedenle pg_dump, pgAgent, staging tabloları, log tabloları ve doğrulama fonksiyonları kullanılmıştır.

1.1 Projenin Amacı

- Veritabanı için tam, artık ve fark yedekleme stratejileri oluşturmak
- Yedekleme işlemlerini belirli zaman aralıklarında otomatik çalıştırmak
- Kazayla veri silme, tablo düşürme ve veri bozulması gibi felaket senaryolarını test etmek
- Belirli bir zamana geri dönüş mantığını point-in-time recovery ile simüle etmek
- Alınan yedeklerin doğruluğunu satır sayısı, checksum ve restore testleriyle kontrol etmek

1.2 Kullanılan Araçlar ve Teknolojiler

Araç	Sürüm	Kullanım Amacı
PostgreSQL	17	Ana veritabanı yönetim sistemi
DBeaver	24.x	SQL geliştirme ve test ortamı
pgAgent	4.x	Yedekleme işlerini zamanlamak
Northwind DB	—	Örnek ilişkisel veritabanı
pg_dump	—	Tam veritabanı yedeği almak
PL/pgSQL	—	Fonksiyon, prosedür ve otomasyon mantığı yazmak

Tablo 1: Kullanılan araçlar ve teknolojiler

2 Sistem Mimarisi ve Altyapı Tasarımı

Projede yedekleme ve kurtarma işlemlerini merkezi olarak yönetmek için `recovery_mgmt` isimli özel bir şema oluşturulmuştur. Bu şema altında yedekleme stratejileri, yedek kayıtları, felaket senaryoları, doğrulama sonuçları, point-in-time recovery logları ve satır sayısı snapshot bilgileri tutulmaktadır.

2.1 Şema ve Tablo Yapısı

Tablo	Şema	Açıklama
backup_strategy	recovery_mgmt	FULL, INCREMENTAL ve DIFFERENTIAL stratejilerini tutar
backup_record	recovery_mgmt	Her yedekleme işleminin log kaydını tutar
disaster_scenario	recovery_mgmt	Felaket senaryolarını ve kurtarma sonuçlarını saklar
backup_verification	recovery_mgmt	Yedek doğrulama testlerinin sonuçlarını tutar
pitr_log	recovery_mgmt	Point-in-time recovery işlemlerini kayıt altına alır
row_count_snapshot	recovery_mgmt	Yedek anındaki tablo satır sayılarını saklar

Tablo 2: recovery_mgmt şeması tablo yapısı

2.2 Altyapı Kurulumu

```
CREATE SCHEMA IF NOT EXISTS recovery_mgmt;  
  
CREATE TABLE IF NOT EXISTS recovery_mgmt.backup_strategy (  
    strategy_id SERIAL PRIMARY KEY,  
    strategy_name VARCHAR(50) NOT NULL,  
    description TEXT,  
    frequency_hours INT NOT NULL,  
    retention_days INT NOT NULL,  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

2.3 backup_record Tablosu

Bu tablo, alınan her yedek için durum, dosya yolu, başlangıç-bitiş zamanı, checksum ve hata mesajı gibi bilgileri saklar.

```
CREATE TABLE IF NOT EXISTS recovery_mgmt.backup_record (  
    record_id          SERIAL PRIMARY KEY,  
    strategy_id        INT REFERENCES recovery_mgmt.  
        backup_strategy(strategy_id),  
    backup_type        VARCHAR(20) NOT NULL,  
    parent_full_id     INT,  
    db_name            VARCHAR(100) NOT NULL,  
    backup_path        TEXT NOT NULL,  
    started_at         TIMESTAMP NOT NULL DEFAULT NOW(),  
    finished_at        TIMESTAMP,  
    status             VARCHAR(20) NOT NULL DEFAULT 'RUNNING',  
    row_count_snap     BIGINT,  
    file_size_bytes    BIGINT,  
    checksum           TEXT,  
    error_message      TEXT,  
    taken_by          VARCHAR(100) DEFAULT CURRENT_USER  
);
```

3 Yedekleme Stratejileri

Projede üç temel yedekleme stratejisi tanımlanmıştır: tam yedek, artık yedek ve fark yedek.

3.1 Strateji Tanımları

Yedek Tipi	Sıklık	Saklama	Açıklama
FULL	168 saat	30 gün	Tüm veritabanının eksiksiz yedeği alınır
INCREMENTAL	6 saat	7 gün	Son yedekten sonra değişen veriler alınır
DIFFERENTIAL	24 saat	14 gün	Son tam yedekten sonra değişen tüm veriler alınır

Tablo 3: Yedekleme stratejileri

```
INSERT INTO recovery_mgmt.backup_strategy
    (strategy_name, description, frequency_hours, retention_days
    )
VALUES
    ('FULL', 'Tum veritabaninin eksiksiz yedeği.', 168, 30),
    ('INCREMENTAL', 'Son yedekten bu yana degisen verilerin
        yedeği.', 6, 7),
    ('DIFFERENTIAL', 'Son FULL yedekten bu yana degisen tum
        verilerin yedeği.', 24, 14)
ON CONFLICT DO NOTHING;
```

4 Yardımcı Fonksiyonlar

4.1 get_strategy_id() Fonksiyonu

Aktif olan yedekleme stratejisinin id bilgisini döndürür.

```
CREATE OR REPLACE FUNCTION recovery_mgmt.get_strategy_id(p_name
    VARCHAR)
RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE
    v_id INT;
BEGIN
    SELECT strategy_id INTO v_id
    FROM recovery_mgmt.backup_strategy
    WHERE strategy_name = p_name AND is_active = TRUE;

    IF v_id IS NULL THEN
        RAISE EXCEPTION 'Strateji bulunamadi veya pasif: %',
            p_name;
    END IF;

    RETURN v_id;
END;
$$;
```

4.2 open_backup_record() Fonksiyonu

Yedekleme işlemi başlamadan önce backup_record tablosuna RUNNING durumunda kayıt açar.

```
CREATE OR REPLACE FUNCTION recovery_mgmt.open_backup_record(
    p_type      VARCHAR,
    p_db        VARCHAR,
    p_path      TEXT,
```



```

        p_parent_id INT DEFAULT NULL
    )
    RETURNS INT
    LANGUAGE plpgsql AS $$
    DECLARE
        v_id          INT;
        v_strat_id INT;
    BEGIN
        v_strat_id := recovery_mgmt.get_strategy_id(p_type);

        INSERT INTO recovery_mgmt.backup_record
            (strategy_id, backup_type, parent_full_id, db_name,
             backup_path, status)
        VALUES
            (v_strat_id, p_type, p_parent_id, p_db, p_path, 'RUNNING
              ')
        RETURNING record_id INTO v_id;

        RETURN v_id;
    END;
    $$;

```

4.3 take_row_snapshot() Fonksiyonu

Northwind veritabanındaki kritik tabloların yedek anındaki satır sayılarını kaydeder. Bu bilgiler daha sonra doğrulama testlerinde kullanılır.

```

CREATE OR REPLACE FUNCTION recovery_mgmt.take_row_snapshot(
    p_record_id INT)
    RETURNS VOID
    LANGUAGE plpgsql AS $$
    BEGIN
        INSERT INTO recovery_mgmt.row_count_snapshot
            (record_id, table_name, row_count)
        SELECT p_record_id, 'customers', COUNT(*) FROM public.

```

```
        customers
UNION ALL
SELECT p_record_id, 'orders', COUNT(*) FROM public.orders
UNION ALL
SELECT p_record_id, 'order_details', COUNT(*) FROM public.
        order_details
UNION ALL
SELECT p_record_id, 'products', COUNT(*) FROM public.
        products
UNION ALL
SELECT p_record_id, 'suppliers', COUNT(*) FROM public.
        suppliers;
END;
$$;
```

5 Tam Yedekleme

Tam yedekleme, veritabanının tamamının pg_dump ile custom formatta alınmasıdır. Diğer yedek türleri için referans noktası oluşturur.

```
CREATE OR REPLACE FUNCTION recovery_mgmt.take_full_backup(  
    p_db VARCHAR DEFAULT 'northwind'  
)  
RETURNS TABLE (  
    record_id INT,  
    status VARCHAR,  
    backup_path TEXT,  
    message TEXT  
)  
LANGUAGE plpgsql AS $$  
DECLARE  
    v_path TEXT;  
    v_cmd TEXT;  
    v_record_id INT;  
BEGIN  
    v_path := '/tmp/dr_backups/' || p_db  
            || '_FULL_' || TO_CHAR(NOW(), 'YYYYMMDD_HH24MISS')  
            || '.dump';  
  
    v_record_id := recovery_mgmt.open_backup_record('FULL', p_db  
        , v_path);  
  
    PERFORM recovery_mgmt.take_row_snapshot(v_record_id);  
  
    BEGIN  
        v_cmd := 'pg_dump -U postgres -Fc -d ' || p_db || ' -f '  
                || v_path;  
        EXECUTE format('COPY (SELECT 1) TO PROGRAM %L', v_cmd);  
  
        UPDATE recovery_mgmt.backup_record
```

```

        SET finished_at = NOW(), status = 'SUCCESS'
        WHERE record_id = v_record_id;

    RETURN QUERY SELECT v_record_id, 'SUCCESS'::VARCHAR,
                        v_path, 'Tam yedek basariyla alindi.'
                        '::TEXT;

EXCEPTION WHEN OTHERS THEN
    UPDATE recovery_mgmt.backup_record
    SET finished_at = NOW(), status = 'FAILED',
        error_message = SQLERRM
    WHERE record_id = v_record_id;

    RETURN QUERY SELECT v_record_id, 'FAILED'::VARCHAR,
                        v_path, ('Hata: ' || SQLERRM)::TEXT;

END;
END;
$$;

```

6 Artık ve Fark Yedekleme

6.1 Artık Yedekleme

PostgreSQL'de gerçek blok seviyesinde incremental backup WAL arşivleme ile yapılabilir. Bu projede artık yedekleme mantığı uygulama seviyesinde simüle edilmiştir. Son tam yedekten sonra değişen orders kayıtları JSON formatında staging tablosuna alınmaktadır.

```
CREATE TABLE IF NOT EXISTS recovery_mgmt.incremental_staging (  
    staging_id    SERIAL PRIMARY KEY,  
    record_id     INT REFERENCES recovery_mgmt.backup_record(  
        record_id),  
    source_table  VARCHAR(100) NOT NULL,  
    row_data      JSONB NOT NULL,  
    captured_at   TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

6.2 Fark Yedekleme

Fark yedekleme, son tam yedekten sonra değişen tüm verileri kapsar. Bu projede orders ve order_details tablolarındaki değişiklikler staging tablosuna kaydedilmiştir.

```
CREATE TABLE IF NOT EXISTS recovery_mgmt.differential_staging (  
    staging_id    SERIAL PRIMARY KEY,  
    record_id     INT REFERENCES recovery_mgmt.backup_record(  
        record_id),  
    source_table  VARCHAR(100) NOT NULL,  
    row_data      JSONB NOT NULL,  
    captured_at   TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

6.3 Artık ve Fark Yedek Karşılaştırması

Özellik	Artık Yedek	Fark Yedek
Referans	Son FULL veya son INCREMENTAL yedek	Son FULL yedek
Boyut	Daha küçüktür	Daha büyüktür
Kurtarma	Tüm yedek zinciri gerekir	FULL + son DIFFERENTIAL yeterlidir
Kullanım	Sık aralıklı yedeklerde avantajlıdır	Daha hızlı kurtarma gereken durumlarda avantajlıdır

Tablo 4: Artık ve fark yedekleme karşılaştırması

7 Otomasyon — pgAgent Job Tanımlamaları

Yedekleme işlemlerinin manuel çalıştırılmasına gerek kalmaması için pgAgent üzerinde üç farklı zamanlanmış iş tanımlanmıştır.

7.1 Job Özet Tablosu

Job Adı	Yedek Tipi	Zamanlama	Açıklama
DR_Weekly_Full_Backup	FULL	Pazar 01:00	Haftalık tam yedek
DR_Incremental_Backup	INCREMENTAL	6 saatte bir	Artık yedek
DR_Differential_Backup	DIFFERENTIAL	Her gece 23:00	Günlük fark yedeği

Tablo 5: Tanımlanan pgAgent job'ları

7.2 Haftalık Tam Yedek Job

```
INSERT INTO pgagent.pga_job
    (jobclid, jobname, jobdesc, jobhostagent, jobenabled)
VALUES (
    1,
    'DR_Weekly_Full_Backup',
    'Her Pazar 01:00 tam yedek - Proje2',
    '',
    TRUE
);
```

7.3 Artık Yedek Job

Bu job en son başarılı FULL yedeği bulur ve ona bağlı olarak incremental backup fonksiyonunu çalıştırır.

```
SELECT record_id
FROM recovery_mgmt.backup_record
WHERE backup_type = 'FULL' AND status = 'SUCCESS'
ORDER BY started_at DESC
LIMIT 1;
```

7.4 Fark Yedek Job

Bu job her gece 23:00'te son FULL yedeği referans alarak fark yedeği oluşturur.

```
PERFORM recovery_mgmt.take_differential_backup(v_full_id);
```


8 Felaketten Kurtarma Senaryoları

Projede üç farklı felaket senaryosu ele alınmıştır.

8.1 Senaryo A: Kazayla Veri Silme

Bu senaryoda 1997 yılına ait sipariş kayıtları yanlışlıkla silinmiş gibi simüle edilmiştir.

```
DELETE FROM public.orders
WHERE EXTRACT(YEAR FROM order_date) = 1997;
```

Silinen verilerin geri yüklenmesi için incremental staging tablosundaki en güncel kayıtlar kullanılmıştır.

```
WITH son_yedek AS (
    SELECT DISTINCT ON (s.row_data->>'order_id')
        s.row_data
    FROM recovery_mgmt.incremental_staging s
    JOIN recovery_mgmt.backup_record r ON s.record_id = r.
        record_id
    WHERE s.source_table = 'orders'
        AND r.status = 'SUCCESS'
    ORDER BY s.row_data->>'order_id', s.captured_at DESC
)
INSERT INTO public.orders (...)
SELECT ...
FROM son_yedek
ON CONFLICT (order_id) DO NOTHING;
```

8.2 Senaryo B: DROP TABLE Simülasyonu

Bu senaryoda products tablosunun yanlışlıkla silinmesi ele alınmıştır. Güvenli test için tablo önce products_backup_snap isimli yedek tabloya kopyalanmıştır.

```
CREATE TABLE IF NOT EXISTS recovery_mgmt.products_backup_snap
AS SELECT * FROM public.products;
```

```
-- DROP TABLE public.products;  
-- CREATE TABLE public.products AS  
-- SELECT * FROM recovery_mgmt.products_backup_snap;
```

8.3 Senaryo C: Veri Bozulması

Bu senaryoda ürün fiyatlarının hatalı bir UPDATE sorgusu ile sıfırlandığı varsayılmıştır.

```
UPDATE public.products SET unit_price = 0;  
  
UPDATE public.products p  
SET unit_price = snap.unit_price  
FROM recovery_mgmt.products_backup_snap snap  
WHERE p.product_id = snap.product_id;
```

9 Point-in-Time Recovery Simülasyonu

Point-in-time recovery, veritabanını belirli bir zaman noktasındaki haline döndürme işlemidir. Bu projede PITR mantığı, hedef zamana en yakın başarılı yedek kaydının bulunması ve staging verileri üzerinden ilgili tabloya ait kayıtların sayılmasıyla simüle edilmiştir.

```
CREATE OR REPLACE FUNCTION recovery_mgmt.pitr_restore(  
    p_target_time    TIMESTAMP ,  
    p_table_name     VARCHAR  
)  
RETURNS TABLE (  
    pitr_id          INT ,  
    rows_found       BIGINT ,  
    status           VARCHAR ,  
    message          TEXT  
)  
LANGUAGE plpgsql AS $$  
DECLARE  
    v_pitr_id        INT ;  
    v_backup_id      INT ;  
    v_row_cnt        BIGINT ;  
BEGIN  
    SELECT record_id INTO v_backup_id  
    FROM recovery_mgmt.backup_record  
    WHERE status = 'SUCCESS'  
        AND started_at <= p_target_time  
    ORDER BY started_at DESC  
    LIMIT 1 ;  
  
    INSERT INTO recovery_mgmt.pitr_log  
        (target_time , used_backup_id , recovery_table)  
    VALUES (p_target_time , v_backup_id , p_table_name)  
    RETURNING pitr_id INTO v_pitr_id ;
```

```

SELECT COUNT(*) INTO v_row_cnt
FROM recovery_mgmt.incremental_staging s
WHERE s.record_id = v_backup_id
      AND s.source_table = p_table_name
      AND s.captured_at <= p_target_time;

UPDATE recovery_mgmt.pitr_log
SET finished_at = NOW(),
    rows_recovered = v_row_cnt,
    status = 'COMPLETED'
WHERE pitr_id = v_pitr_id;

RETURN QUERY SELECT v_pitr_id, v_row_cnt,
                    'COMPLETED'::VARCHAR,
                    'PITR tamamlandi.'::TEXT;

END;
$$;

```

9.1 PITR Testi

```

SELECT * FROM recovery_mgmt.pitr_restore(
    NOW() - INTERVAL '3 hours',
    'orders'
);

```

10 Yedek Doğrulama Testleri

Alınan yedeklerin güvenilirliğini test etmek için üç farklı doğrulama mekanizması kullanılmıştır.

10.1 ROW COUNT Testi

Yedek anında alınan tablo satır sayısı snapshot'ı ile mevcut satır sayıları karşılaştırılır.

```
SELECT * FROM recovery_mgmt.verify_row_count(v_record_id);
```

10.2 CHECKSUM Testi

Belirli bir tablonun verileri metin olarak birleştirilir ve MD5 hash değeri hesaplanır. Kayıtlı checksum değeri ile güncel checksum karşılaştırılır.

```
SELECT * FROM recovery_mgmt.verify_checksum(  
    p_record_id := 1,  
    p_table := 'orders'  
);
```

10.3 RESTORE TEST

Yedekteki staging verisinin gerçekten oluşup oluşmadığı kontrol edilir. Kayıt sayısı sıfırdan büyükse test başarılı kabul edilir.

```
SELECT * FROM recovery_mgmt.verify_restore_test(1);
```

10.4 Doğrulama Testleri Özeti

Test Türü	Amaç	Beklenen Sonuç
ROW_COUNT	Tablo satır sayılarının korunup korunmadığını kontrol eder	PASS
CHECKSUM	Veri bütünlüğünü hash değeriyle kontrol eder	PASS
RESTORE_TEST	Yedek staging verisinin geri yüklenebilirliğini test eder	PASS

Tablo 6: Yedek doğrulama testleri

11 Test ve Demo Sorguları

11.1 Yedek Alma Testleri

```
-- Tam yedek al
SELECT * FROM recovery_mgmt.take_full_backup('northwind');

-- En son FULL yedek id'sini bul
SELECT record_id
FROM recovery_mgmt.backup_record
WHERE backup_type = 'FULL' AND status = 'SUCCESS'
ORDER BY started_at DESC
LIMIT 1;

-- Art k yedek al
SELECT * FROM recovery_mgmt.take_incremental_backup(1);

-- Fark yedek al
SELECT * FROM recovery_mgmt.take_differential_backup(1);
```

11.2 Felaket Senaryosu Kontrolleri

```
SELECT * FROM recovery_mgmt.disaster_scenario
ORDER BY simulated_at DESC;

SELECT COUNT(*) AS siparis_sayisi
FROM public.orders;

SELECT
    COUNT(*) FILTER (WHERE unit_price > 0) AS
        gecerli_fiyat_sayisi,
    COUNT(*) FILTER (WHERE unit_price = 0) AS bozuk_kalan
FROM public.products;
```

11.3 Doğrulama Sonuçları

```
SELECT *  
FROM recovery_mgmt.backup_verification  
ORDER BY verified_at DESC;  
  
SELECT *  
FROM recovery_mgmt.pitr_log  
ORDER BY started_at DESC;
```


12 Sonuç

Bu projede Northwind veritabanı için kapsamlı bir yedekleme ve felaketten kurtarma altyapısı tasarlanmıştır. Sistem; tam, artık ve fark yedekleme stratejilerini desteklemekte, pgAgent ile zamanlanmış yedekleme işleri oluşturmakta ve farklı felaket senaryolarında verinin geri getirilebilmesini sağlamaktadır.

Kazayla silinen kayıtların geri yüklenmesi, tablo düşürme senaryosunda yedek tablodan kurtarma yapılması ve veri bozulması durumunda hatalı değerlerin eski haline döndürülmesi test edilmiştir. Ayrıca point-in-time recovery mantığı ile belirli bir zamana ait yedek kaydının bulunması sağlanmıştır.

Son olarak satır sayısı kontrolü, checksum karşılaştırması ve restore testi ile yedeklerin doğruluğu denetlenmiştir. Böylece yalnızca yedek alan değil, aynı zamanda yedeğin güvenilirliğini test eden ve felaket durumunda kurtarma adımlarını belgeleyen bütüncül bir veritabanı kurtarma planı oluşturulmuştur.